

A Practical Guide to OpenCV C/C++ on Ubuntu

Pham Minh Hoang

January 26, 2024

1 Installation and Configuration

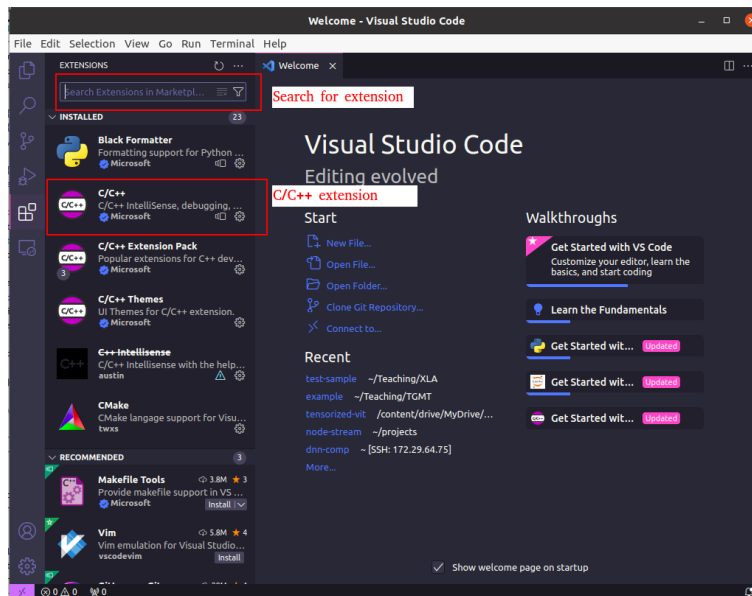
1.1 Installing c++ compiler and build-tools

Run the following command in the terminal to install gcc, g++, gdb, and make.

```
$ sudo apt install build-essential gdb
```

1.2 Installing VSCode and C++ Extension

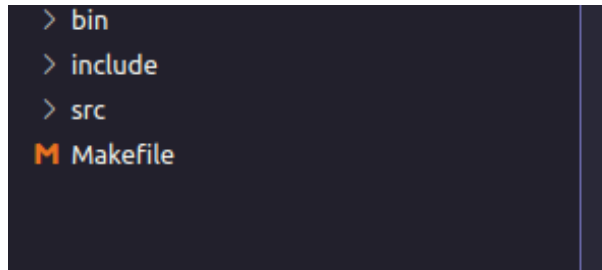
Check out this website and check for instructions to install VSCode on your computer. Once VSCode is installed, open it. You should see the following screen.



Now, to install C++ extension, go to extensions from the activity bar. Search for C++ in the search bar and install C/C++ Extension By Microsoft.

1.3 Setting Up Project Directory

After successfully installing VSCode and the necessary extensions, create a project with the following directory structure



- The *bin* directory will contain the executable after compilation
- The *include* directory will contain all the header files
- The *src* directory will contain all the source files

The *Makefile* is a file located in the project directory to define tasks that will be executed in order. Create a Makefile with the following content in the root of your project directory

```
1 CXX      := g++
2 CXX_FLAGS := -std=c++17 -ggdb
3
4 BIN       := bin
5 SRC       := src
6 INCLUDE   := include
7
8 LIBRARIES := `pkg-config --cflags --libs opencv4`
9 EXECUTABLE := main
10
11 all: $(BIN)/$(EXECUTABLE)
12
13 run: clean all
14     clear
15     ./$(BIN)/$(EXECUTABLE)
16
17 $(BIN)/$(EXECUTABLE): $(SRC)/*.cpp
18     $(CXX) $(CXX_FLAGS) -I$(INCLUDE) $^ -o $@ $(LIBRARIES)
19
20 clean:
21     -rm $(BIN)/*
```

Now that we have our base ready let's add some C++ files. Create a *main* cpp file that print out "Hello, World".

```
$ touch src/main.cpp
```

Add following to *main.cpp*.

Source Code 1:

```
int main()
{
    std::cout << "Hello, World";
    return 0;
}
```

Now, we have all our project all set up. Open a terminal in the project directory and run make.

1.4 Project configuration

1.4.1 Setting up file *"tasks.json"*

We can set up so that by simple pressing Ctrl+Shift+B, VSCode will run the build task. To do this, we need to open and modify *tasks.json* of VSCode

- To open file *tasks.json* in VSCode, press Ctrl+Shift+P, enter *Task: Configure Tasks*
- Copy the following content to *tasks.json*

Source Code 2:

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "build",
      "type": "shell",
      "command": "make",
      "group": {
        "kind": "build",
        "isDefault": true
      },
      "problemMatcher": "$gcc"
    }
  ]
}
```

1.4.2 Setting up gdb as debugger

We have successfully built our code. Now we are going to configure debugger. To debug the program, configure debugger using *launch.json*.

- To open file *launch.json*, press Ctrl+Shift+P, enter *Debug: Add Configuration*, select *gdb*
- Copy the following content to *launch.json*

1.5 Setting up OpenCV

- To install OpenCV, use prebuilt binaries

```
$ sudo apt install libopencv-dev
```
- Open *c_cpp_properties.json* by pressing Ctrl+Shift+P and enter and copy following content

```

{
  // Use IntelliSense to learn about possible attributes.
  // Hover to view descriptions of existing attributes.
  // For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
  "version": "0.2.0",
  "configurations": [
    {
      "name": "(gdb) Launch",
      "type": "cppdbg",
      "request": "launch",
      "program": "${workspaceFolder}/bin/main",
      "args": [
        "-input",
        "images/test.png"
      ],
      "stopAtEntry": false,
      "cwd": "${workspaceFolder}",
      "environment": [],
      "externalConsole": false,
      "MIMode": "gdb",
      "setupCommands": [
        {
          "description": "Enable pretty-printing for gdb",
          "text": "-enable-pretty-printing",
          "ignoreFailures": true
        }
      ],
      "preLaunchTask": "build"
    }
  ]
}

```

launch.json

```

{
  "configurations": [
    {
      "name": "Linux",
      "includePath": [
        "${workspaceFolder}/**",
        "/usr/include/opencv4"
      ],
      "defines": [],
      "compilerPath": "/usr/bin/gcc",
      "cStandard": "gnu17",
      "cppStandard": "gnu++14",
      "intelliSenseMode": "linux-gcc-x64"
    }
  ],
  "version": 4
}

```

c_cpp_properties.json

1.6 Try to run the program

Copy this source code into the main cpp file and try to run it

Source Code 3:

```
#include "stdafx.h"
#include "opencv2/opencv.hpp"
#include "opencv2/highgui/highgui.hpp"
using namespace cv;

int main()
{
    //read the input image from file
    Mat image = imread("D:\\lena.png", CV_LOAD_IMAGE_COLOR);

    //create and set the window name
    namedWindow("Show_Image");

    //show the image on window
    imshow("Show_Image", image);

    //close the window
    waitKey(0);
    return 0;
}
```

The installation and project configuration can be finished if the program shows the image successfully

2 Practical guide OpenCV

In OpenCV, an image (or matrix) is stored by a *Mat* object

2.1 Mat structure

Define the *Mat* structure

```

class CV_EXPORTS Mat
{
public:
    // ... a lot of methods ...
    ...

    /*! includes several bit-fields:
        - the magic signature
        - continuity flag
        - depth
        - number of channels
    */
    int flags;
    ///! the array dimensionality, >= 2
    int dims;
    ///! the number of rows and columns or (-1, -1) when the array has more than 2 dimensions
    int rows, cols;
    ///! pointer to the data
    uchar* data;

    ///! pointer to the reference counter;
    // when array points to user-allocated data, the pointer is NULL
    int* refcount;

    // other members
    ...
};

```

Source Code 4: Matrix declaration

```

//declare an empty matrix
Mat::Mat();

//declare an matrix with the number of rows and columns
Mat::Mat(int rows, int cols, int type);

//declare an matrix with its size
Mat::Mat(Size size, int type);

//declare an matrix with the default value
Mat::Mat(int rows, int cols, int type, const Scalar& s);

```

Explanation of parameters

- *rows*: the number of rows is equivalent to the image height
- *cols*: the number of columns is equivalent to the image width
- *type*: data type of matrix are represented in the belowed form

CV_<#bits><data type><#channels>

- *#bits*: the number of bits in a channel
 - *data type*: data type of each element in the matrix (UC: unsigned char, SC: signed, FC: float)
 - *#channels*: the number of channels
- For example
 - *CV_8UC1*: 1 channel, 8 bit per channel, data type: unsigned char, grayscale image
 - *CV_8UC3*: 3 channel, 8 bit per channel, data type: unsigned char, color image
 - *CV_16SC1*: 1 channel, 16 bit per channel, data type: signed short
 - *CV_32FC1*: 1 channel, 32 bit per channel, data type: float
 - *CV_64FC1*: 1 channel, 64 bit per channel, data type: double

Source Code 5: Example of matrix declaration

```
//Matrix a with size of 100 x 100, 1 channel
//data type of each element: unsigned char, grayscale image
Mat a = Mat(100, 100, CV_8UC1);

//Matrix a with size of 200 x 200, 3 channels,
//data type of each element: unsigned char, RGB image
Mat a = Mat(200, 200, CV_8UC3, Scalar(0));

//Matrix a with size of 120 x 240, 1 channel,
//data type of each element: 32 bit float
Mat a = Mat(Size(120, 240), CV_32FC1);

//Matrix a with size of 240 x 120, 1 channel,
//data type of each element: 64 bit float (double)
Mat a = Mat(Size(120, 240), CV_64FC1);
```

2.2 Some basis functions

Load image from file

Source Code 6:

```
//load a color image
Mat image = imread("filename.jpg", CV_LOAD_IMAGE_COLOR);

//load a grayscale image
Mat image = imread("filename.jpg", CV_LOAD_IMAGE_GRAYSCALE);
```

Save image

Source Code 7:

```
imwrite("filename.jpg", image);
```

Display image on window

Source Code 8:

```
//create a window and set a name "Show image"
namedWindow("Show_Image");

//display image on the window "Show Image"
imshow("Show_Image", image);

//wait for user to close the window
waitKey(0);
```

2.3 Pixel processing

OpenCV stores a grayscale image matrix [1]

	Column 0	Column 1	Column ...	Column m
Row 0	0,0	0,1	...	0, m
Row 1	1,0	1,1	...	1, m
Row ...	...,0	...,1	...	..., m
Row n	n,0	n,1	n,...	n, m

OpenCV stores a color image according to BGR order (the first color is Blue, the next color

is Green, and the last one is Red) [1]

	Column 0			Column 1			Column ...			Column m		
Row 0	0,0	0,0	0,0	0,1	0,1	0,1	...	...	...	0, m	0, m	0, m
Row 1	1,0	1,0	1,0	1,1	1,1	1,1	...	...	...	1, m	1, m	1, m
Row ...	...,0	...,0	...,0	...,1	...,1	...,1	...	...	...	..., m	..., m	..., m
Row n	n,0	n,0	n,0	n,1	n,1	n,1	n,...	n,...	n,...	n, m	n, m	n, m

To get a pixel, we can use three methods

Source Code 9: Using at() function

```
void method_1(cv::Mat &image, int n) {
    //get image width
    int width = image.cols;
    //get image height
    int height = image.rows;
    for (int y = 0; y < height; y++) {
        for(int x = 0; x < width; x++) {
            //for grayscale image
            if (image.channels() == 1) {
                image.at<uchar>(x, y) = 255;
            }
            //for color image
            else if (image.channels() == 3) {
                cv::Vec3b& pixel = image.at<cv::Vec3b>(x, y);
                //get the first color in this pixel (blue)
                pixel[0] = 255;
                //get the second color in this pixel (green)
                pixel[1] = 255;
                //get the last color in this pixel (red)
                pixel[2] = 255;
            }
        }
    }
}
```

Source Code 10: Using the pointer to the first element of each row

```

void method_2(cv::Mat &image, int n) {
    int width = image.cols, height = image.rows;
    int nChannels = image.channels();
    for (int y = 0; y < height; y++) {
        //get the pointer to the first element of the y-th row
        uchar* pRow = image.ptr<uchar>(y);
        for(int x = 0; x < width; x++, pRow += nChannels) {
            //get the first color in this pixel (blue)
            pRow[0] = 255;
            //get the second color in this pixel (green)
            pRow[1] = 255;
            //get the last color in this pixel (red)
            pRow[2] = 255;
        }
    }
}

```

According to [2], the class *Mat* represents an n-dimensional dense numerical single-channel or multi-channel array. It can be used to store real or complex-valued vectors and matrices, grayscale or color images, voxel volumes, vector fields, point clouds, tensors, histograms (though, very high-dimensional histograms may be better stored in a *SparseMat*). The data layout of the array *M* is defined by the array *M.step[]*, so that the address of element $(i_0, \dots, i_{M.dims-1})$, where $0 \leq i_k < M.size[k]$, is computed as:

$$addr(M_{i_0, \dots, i_{M.dims-1}}) = M.data + M.step[0]i_0 + M.step[1]i_1 + \dots + M.step[M.dims-1]i_{M.dims-1}$$

In case of a 2-dimensional array, the above formula is reduced to:

$$addr(M_{i,j}) = M.data + M.step[0]i + M.step[1]j$$

where *M.step[0]* is also *widthStep*, which is the byte distance between two consecutive pixels on the same column, and *M.step[1]* is the number of channels

Note that $M.step[i] \geq M.step[i+1]$ (in fact, $M.step[i] \geq M.step[i+1]M.size[i+1]$). This means that 2-dimensional matrices are stored row-by-row, 3-dimensional matrices are stored plane-by-plane, and so on. *M.step[M.dims-1]* is minimal and always equal to the element size *M.elemSize()*.

Source Code 11: Using the pointer to manage the image data

```
void method_3(cv::Mat &image, int n) {
    int width = image.cols, height = image.rows;
    int widthStep = image.steps[0];
    int nChannels = image.steps[1];
    uchar* pData = (uchar*)image.data;
    for (int y = 0; y < height; y++, pData += widthStep) {
        uchar* pRow = pData;
        for(int x = 0; x < width; x++, pRow += nChannels) {
            //get the first color in this pixel (blue)
            pRow[0] = 255;
            //get the second color in this pixel (green)
            pRow[1] = 255;
            //get the last color in this pixel (red)
            pRow[2] = 255;
        }
    }
}
```

2.4 GUI programming

To add a trackbar, we can use the function *createTrackbar* [3], [4]

```
int createTrackbar(const string& trackbarname, const string& winname, int* value,
int count, TrackbarCallback onChange = 0, void* userdata = 0)
```

Explanation of parameters

- trackbarname – Name of the created trackbar.
- winname – Name of the window that will be used as a parent of the created trackbar.
- value – Optional pointer to an integer variable whose value reflects the position of the slider. Upon creation, the slider position is defined by this variable.
- count – Maximal position of the slider. The minimal position is always 0.
- onChange – Pointer to the function to be called every time the slider changes position. This function should be prototyped as `void Foo(int,void*);`, where the first parameter is the trackbar position and the second parameter is the user data (see the next parameter). If the callback is the NULL pointer, no callbacks are called, but only value is updated.

- `userdata` – User data that is passed as is to the callback. It can be used to handle trackbar events without using global variables.

In the following OpenCV example, we have added a trackbar to change the value. And a callback function is implemented for this trackbar.

Source Code 12: Using a trackbar

```
void CallbackFunction(int pos, void *userData)
{
    //cast userData to value
    int valueFromUser = *( static_cast<int*>(userData) );
    ...
}

int main(int argc, char** argv)
{
    // Read original image
    src = imread("MyPic.JPG");

    // Create a window
    namedWindow("My_Window", 1);

    int pos = 50;
    int val = 50;

    //Create track bar
    createTrackbar("Trackbar_Name", "My_Window", &pos, 100,
        CallbackFunction, &val);

    imshow("My_Window", src);

    // Wait until user press some key
    waitKey(0);

    return 0;
}
```

References

- [1] The OpenCV Tutorials

- [2] The OpenCV Document page
<https://docs.opencv.org/3.4/index.html>
- [3] OpenCV Tutorial C++
<https://www.opencv-srf.com/2011/11/track-bars.html>
- [4] Learn OpenCV by Examples
<http://opencvexamples.blogspot.com/2013/10/adding-trackbar.html>